ODI SaaS Platform — 智能ODI合规与出海服务管控平台

> 面向跨境出海中国企业的智能境外直接投资（ODI）备案 SaaS 平台，支持 AI 智能预审、多部委电子表单自动生成、项目全生命周期管理。

English | 中文

✨ 核心功能

🏛️ 项目全生命周期管理

- **9阶段状态机**: 意向测算 → 智能预审 → 材料准备 → 发改委备案 → 商务部备案 → 银行外汇登记 → 资金汇出 → 投后管理
- 支持创建、编辑、删除、状态推进
- 境内/境外主体关联管理

🤖 AI 智能预审

- 基于规则引擎的合规校验（目的国、行业、金额等）
- LLM 综合分析，自动生成预审报告（交通灯风险评级）
- **DeepSeek / Kimi / MiniMax** 三大模型自动路由，失败自动降级
- 重试机制（3次指数退避）

📄 AI 报告生成

- 可行性研究报告（feasibility）
- 尽职调查报告（due_diligence）
- 财务数据自动抽取（支持从 PDF 解析）

📋 多部委电子表单导出

- **发改委 XML 格式备案文件** — 结构化导出
- **商务部 Excel 格式备案文件** — 自动填充字段映射
- **字段映射可配置** — 管理后台随时调整导出模板
- 一键打包下载（ZIP）

💳 点数计费系统

- 按实际 LLM Token 消耗计费（每 1000 tokens = 1 点）
- 年费会员免扣点
- 计费流水完整记录

🔐 安全与限流

- JWT Access Token（15分钟）+ Refresh Token（7天）
- RBAC 角色权限（ADMIN / OPERATOR / VIEWER）
- API 限流（登录 20/min，注册 10/min，刷新 30/min）
- 密码 bcrypt 哈希

🏗️ 技术架构

```
┌──────────────────────────────────────────────────────────────────┐
│                        Frontend (Vue 3)                          │
│   Ant Design Vue + Pinia + Vue Router + Vite                     │
└──────────────────────────┬───────────────────────────────────────┘
                           │ REST API (nginx reverse proxy)
┌──────────────────────────▼───────────────────────────────────────┐
│                        Backend (FastAPI + Uvicorn)               │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐       │
│  │ Auth/RBAC    │  │ LLM Router   │  │ Billing Service  │       │
│  │ (JWT+刷新)   │  │ (DeepSeek/   │  │ (Token计费)      │       │
│  │              │  │  Kimi/Mini)  │  │                  │       │
│  └──────────────┘  └──────────────┘  └──────────────────┘       │
│                                                                  │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────┐   │
│  │ Project Service  │  │ Rules Engine     │  │ Export       │   │
│  │ (9阶段状态机)    │  │ (合规规则匹配)   │  │ Engine       │   │
│  │                  │  │                  │  │ (XML/Excel)  │   │
│  └──────────────────┘  └──────────────────┘  └──────────────┘   │
└──────────────────────────────┬───────────────────────────────────┘
                               │ async SQLAlchemy 2.0
┌──────────────────────────────▼───────────────────────────────────┐
│                   PostgreSQL 16 (Async)                          │
│  tenants | users | projects | entities | rules | billing_logs    │
│  llm_configs | export_templates                                  │
└──────────────────────────────────────────────────────────────────┘
```

技术栈

层级	技术
前端	Vue 3.5 + Vite 7 + Ant Design Vue 4 + Pinia + Vue Router 4
后端	FastAPI 0.115 + Uvicorn 0.30 + Python 3.10
数据库	PostgreSQL 16 (async via asyncpg)
ORM	SQLAlchemy 2.0 (async) + Alembic 迁移
AI	DeepSeek / Kimi / MiniMax (OpenAI-compatible API)
导出	lxml (XML) + openpyxl (Excel)
限流	slowapi 0.1.9
重试	tenacity 9.0
部署	Docker Compose + Nginx

🚀 快速启动

环境要求

- Docker & Docker Compose
- Python 3.10+ (本地开发)
- Node.js 22+ (前端开发)

方式一：Docker 一键启动（推荐）

```
# 克隆项目
git clone https://github.com/chenjf2025/ODI.git
cd ODI

# 复制环境变量模板
cp .env.example .env.development

# 启动所有服务
docker-compose up -d

# 验证服务
curl http://localhost/health
# {"status":"healthy"}

# 访问应用
# 前端: http://localhost
# 后端 API: http://localhost:8000
# API 文档: http://localhost:8000/docs
```

方式二：本地开发

后端

```
cd backend

# 创建虚拟环境
python -m venv .venv
source .venv/bin/activate  # Linux/Mac
# .venv\Scripts\activate  # Windows

# 安装依赖
pip install -r requirements.txt

# 配置环境变量
cp ../.env.example .env.development
# 编辑 .env.development 填入必要的 API Key

# 数据库 (Docker)
docker-compose up -d db

# 启动服务
uvicorn app.main:app --reload --port 8000
```

前端

```
cd frontend

# 安装依赖
npm install

# 开发模式
npm run dev
# 生产构建
npm run build
```

⚙️ 环境配置

必需的环境变量

```
# .env.development 或 .env.production

# 数据库
```

```
# 数据库
DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/odi_saas

# JWT 密钥（生产必须修改！）
JWT_SECRET_KEY=your-super-secret-key-min-32-chars
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=15

# LLM API Keys（至少配置一个）
DEEPSEEK_API_KEY=sk-xxxxx
DEEPSEEK_BASE_URL=https://api.deepseek.com

KIMI_API_KEY=sk-xxxxx
KIMI_BASE_URL=https://api.moonshot.cn

MINIMAX_API_KEY=xxxxx
MINIMAX_BASE_URL=https://api.minimax.chat

# CORS（生产环境不要包含 localhost）
CORS_ORIGINS=http://localhost,http://127.0.0.1
```

CORS 配置说明

生产环境 `CORS_ORIGINS` **不应包含** `localhost` 或 `127.0.0.1`，系统启动时会校验并警告。

📚 API 文档

启动后访问: **http://localhost:8000/docs** (Swagger UI)

认证

方法	路径	说明	限流
POST	`/api/auth/register`	用户注册（自动创建租户）	10/min
POST	`/api/auth/login`	登录	20/min
POST	`/api/auth/refresh`	刷新 Token	30/min
GET	`/api/auth/me`	当前用户信息	—

项目

方法	路径	说明	权限
POST	`/api/projects`	创建项目（扣点）	登录
GET	`/api/projects`	项目列表（分页）	登录
GET	`/api/projects/{id}`	项目详情	登录
PUT	`/api/projects/{id}/status`	推进状态	OPERATOR+
DELETE	`/api/projects/{id}`	删除项目	ADMIN

AI 服务

方法	路径	说明	权限
POST	`/api/ai/pre-review`	智能预审（按 Token 计费）	登录

POST	`/api/ai/generate-report`	生成报告（按 Token 计费）	OPERATOR+
POST	`/api/ai/extract-financial`	抽取财务数据	登录

导出

方法	路径	说明	权限
GET	`/api/export/{id}/ndrc-xml`	发改委 XML	OPERATOR+
GET	`/api/export/{id}/mofcom-excel`	商务部 Excel	OPERATOR+
GET	`/api/export/{id}/package`	打包下载	OPERATOR+

管理

方法	路径	说明	权限
GET/POST	`/api/admin/llm-configs`	LLM 配置管理	ADMIN
PUT/DELETE	`/api/admin/llm-configs/{id}`	更新/删除 LLM 配置	ADMIN
POST	`/api/admin/llm-configs/reload`	热更新 LLM 提供商	ADMIN
GET	`/api/admin/export-templates`	导出模板管理	ADMIN
PUT	`/api/admin/export-templates/{type}/{id}`	更新导出字段映射	ADMIN
POST	`/api/admin/export-templates/reorder`	重排导出字段顺序	ADMIN

主体

方法	路径	说明
GET/POST	`/api/entities/domestic`	境内主体 CRUD
GET/POST	`/api/entities/overseas`	境外标的 CRUD

规则

方法	路径	说明	权限
GET	`/api/rules`	合规规则列表	登录
POST	`/api/rules`	创建规则	ADMIN
PUT/DELETE	`/api/rules/{id}`	更新/删除规则	ADMIN

📁 项目结构

```
ODI/
├── backend/
|   ├── app/
|   |   ├── api/                # REST API 路由
|   |   |   ├── auth.py        # 认证（注册/登录/刷新）
|   |   |   ├── projects.py    # 项目 CRUD + 状态流转
|   |   |   ├── entities.py    # 境内外主体管理
|   |   |   ├── ai.py          # AI 预审/报告/抽取
```

```
|   |   |   ├── ai.py          # AI 预审/报告/描述
|   |   |   ├── export.py      # 多部委表单导出
|   |   |   ├── rules.py       # 合规规则 CRUD
|   |   |   ├── admin.py       # LLM 配置管理
|   |   |   ├── admin_export.py # 导出模板管理
|   |   |   ├── tenants.py     # 租户管理
|   |   |   └── upload.py      # 文件上传
|   |   ├── models/            # SQLAlchemy ORM 模型
|   |   |   ├── user.py        # 用户 + 角色枚举
|   |   |   ├── tenant.py       # 租户 + 订阅计划
|   |   |   ├── project.py     # 项目 + 9阶段状态机
|   |   |   ├── entity.py      # 境内外主体
|   |   |   ├── billing.py     # 计费流水
|   |   |   ├── rules.py       # 合规规则引擎
|   |   |   ├── llm_config.py # LLM 提供商配置
|   |   |   └── export_template.py # 导出字段模板
|   |   ├── services/          # 业务逻辑层
|   |   |   ├── ai_service.py          # AI 业务（预审/报告）
|   |   |   ├── billing_service.py     # 计费拦截
|   |   |   ├── project_service.py     # 项目状态机引擎
|   |   |   ├── rules_service.py       # 规则匹配引擎
|   |   |   ├── export_engine.py      # XML/Excel 生成
|   |   |   ├── llm/                   # LLM 集成
|   |   |   |   ├── gateway.py         # 统一网关（工厂模式）
|   |   |   |   ├── router.py         # 任务类型路由调度
|   |   |   |   └── providers/         # DeepSeek / Kimi / MiniMax
|   |   |   └── corporate_info/       # 企业信息查询（企查查/天眼查/百度）
|   |   ├── middleware/
|   |   |   └── auth.py        # JWT 解析 + RBAC 权限装饰器
|   |   ├── schemas/            # Pydantic 请求/响应模型
|   |   ├── utils/              # utc_now() + enum_value()
|   |   ├── exceptions.py       # 统一异常类
|   |   ├── config.py          # Pydantic Settings（含环境变量校验）
|   |   ├── database.py         # async SQLAlchemy 引擎
|   |   └── main.py            # FastAPI 应用入口 + lifespan
|   ├── alembic/               # 数据库迁移
|   |   └── versions/           # 迁移脚本
|   └── requirements.txt
├── frontend/
|   ├── src/
|   |   ├── api/index.js       # Axios API 封装
|   |   ├── stores/user.js     # Pinia 用户状态
|   |   ├── router/index.js     # Vue Router（含路由守卫）
|   |   ├── layouts/MainLayout.vue  # 主布局
|   |   └── views/             # 页面组件
|   |       ├── Login.vue
|   |       ├── Dashboard.vue
|   |       ├── Projects.vue / ProjectDetail.vue
|   |       ├── DomesticEntities.vue / OverseasEntities.vue
|   |       ├── AIReports.vue
|   |       ├── Rules.vue
|   |       ├── Billing.vue
|   |       └── LLMConfig.vue
|   ├── nginx.conf              # 前端 Nginx 配置
|   └── package.json
├── docker-compose.yml           # 一键启动
├── Dockerfile.backend / .frontend
├── nginx.conf                   # 反向代理配置
└── .env.example                # 环境变量模板
```

🔧 开发指南

添加新的 LLM 提供商

```
# 1. 在 providers/ 下创建新的 Provider 类
```

```python
# backend/app/services/llm/providers/myprovider.py

class MyProvider(BaseLLMProvider):
    @property
    def provider_name(self) -> str:
        return "myprovider"

    async def chat_completion(self, messages, temperature, max_tokens, **kwargs) -> LLMResponse:
        # 实现调用逻辑...

        return LLMResponse(content=..., model=..., provider=self.provider_name, usage={...})

# 2. 在 main.py 的 PROVIDER_CLASS_MAP 注册
PROVIDER_CLASS_MAP = {
    "deepseek": DeepSeekProvider,
    "kimi": KimiProvider,
    "minimax": MiniMaxProvider,
    "myprovider": MyProvider,  # 新增
}

# 3. 在管理后台添加配置或在环境变量中配置
```

添加新的合规规则

```
# 在管理后台 API 或直接插入数据库
# RulesEngine 表字段: target_country, industry_code, risk_level, rule_type,
#                     target_value, description, trigger_action
```

修改导出字段映射

导出字段（发改委 XML 标签、商务部 Excel 表头）可通过管理后台 API 实时修改，无需重启服务：

```
GET  /api/admin/export-templates?template_type=NDRC
PUT  /api/admin/export-templates/NDRC/{template_id}
POST /api/admin/export-templates/reorder  # 重排字段顺序
```

🐛 常见问题

Q: API 返回 401 Unauthorized
A: 检查 Token 是否过期（Access Token 15分钟有效），或请求头 `Authorization: Bearer <token>` 格式是否正确。

Q: LLM 调用全部失败
A: 确认已在管理后台或环境变量中正确配置了至少一个 LLM Provider 的 API Key 和 Base URL。

Q: 导出 Excel 文件打开报错
A: 确保 `openpyxl` 版本兼容，Docker 环境使用已锁定版本 `3.1.5`。

Q: 余额充足但仍然报 INSUFFICIENT_FUNDS
A: 检查租户订阅计划，年费会员需确保 `subscription_expiry` 未过期。

📄 License

MIT License

English

ODI SaaS Platform — Intelligent Overseas Direct Investment Compliance Platform

> An AI-powered SaaS for Chinese enterprises' overseas direct investment (ODI) filing management, featuring intelligent compliance review, multi-ministry e-form generation, and project lifecycle management.

Key Features

- **Project Lifecycle**: 9-stage state machine (pre-review → NDRC filing → MOFCOM filing → banking → fund remittance → post-investment)
- **AI Pre-Review**: Rule-based compliance checking + LLM-powered comprehensive analysis with risk traffic-light rating
- **AI Reports**: Feasibility studies, due diligence reports, financial data extraction
- **Multi-Ministry Export**: NDRC XML + MOFCOM Excel with configurable field mapping
- **Token-Based Billing**: Actual LLM token consumption (1 credit per 1000 tokens)
- **Security**: JWT auth (15min access + 7day refresh) + RBAC + API rate limiting

Tech Stack

Layer	Technology
Frontend	Vue 3.5 + Vite 7 + Ant Design Vue 4 + Pinia
Backend	FastAPI 0.115 + Uvicorn + Python 3.10
Database	PostgreSQL 16 (async via asyncpg)
ORM	SQLAlchemy 2.0 (async) + Alembic
AI	DeepSeek / Kimi / MiniMax (OpenAI-compatible)
Export	lxml (XML) + openpyxl (Excel)
Rate Limiting	slowapi 0.1.9
Retry	tenacity 9.0
Deploy	Docker Compose + Nginx

Quick Start

```
git clone https://github.com/chenjf2025/ODI.git
cd ODI
cp .env.example .env.development
# Edit .env.development with your API keys
docker-compose up -d
# Frontend: http://localhost
# API: http://localhost:8000
# Docs: http://localhost:8000/docs
```

API Documentation

Available at **http://localhost:8000/docs** (Swagger UI) after startup.